

Enhancing Flexibility and Safety of eBPF Programs through Verifier-Cooperative Instrumentation

PI: Ryan Huang, Associate Professor, University of Michigan

Motivation

eBPF has become essential to easily extend kernel at runtime for observability [5, 4], networking [7], profiling, security, *etc.* This vibrant ecosystem is gated by the static verifier, which requires a careful balance—the verifier must *conservatively* over-approximate execution paths to guarantee memory safety, bounded loops, and valid helper usage, yet it must be sufficiently *permissive* to support legitimate eBPF use cases. With the increasingly wide and evolving spectrum of eBPF programs [19, 13, 20], this balance is breaking down. The verifier faces significant challenges:

- **False rejections that block real use cases.** Developers routinely encounter rejections [17, 18, 10, 12, 11, 3, 1, 8, 6, 2] for programs that are semantically safe but difficult for the verifier to reason about precisely, *e.g.*, data-dependent loop bounds, indirect or masked array indexing into maps, staged helper invocation patterns, stack aliasing across control paths, and programs that trigger inflated instruction-count estimates. Manual workarounds (tail calls, mask insertion, code cloning, *etc.*) bloat programs and degrade maintainability.
- **Subtle unsafety that slips through.** The verifier cannot guard against all runtime misbehavior, including its own bugs [16, 12, 10, 17, 15]. Gaps in stack initialization tracking, imprecise pointer provenance, helper argument misclassification, *etc.*, have led to exploits, *e.g.*, pointer leaks and privilege escalation, after a program was deemed safe. As eBPF evolves with new helper APIs, map types, among others, the attack surface grows faster than static reasoning can keep pace.
- **Ad-hoc and brittle patching.** The verifier codebase has accumulated ad-hoc special cases over years. Each new kernel feature, helper function, or calling convention risks growth in verification rules and state-space complexity. The tight coupling between verifier internals and kernel complicates maintenance and makes portability brittle.

These tensions create a widening **programmability–safety** gap. Despite existing efforts [10, 17] that improve the verifier, the challenges are inherent due to fundamental limitations of static analysis. Static verification alone cannot scale to meet both developer demands and safety guarantees without incurring untenable complexity or hurting programmability. Simply relaxing the verifier rules to unlock more programs would undermine kernel safety guarantees.

A promising alternative path is to augment static verification with **runtime enforcement**. Recent efforts have illustrated some benefits of this path. For example, KFlex [9] enables dynamic loops with termination checks; SafeBPF [15, 14] uses dynamic sandboxing to achieve stronger memory safety. However, they each target narrow scenarios and require fragile manual bytecode or binary rewriting for instrumentation. Thus, they require high manual effort, lack correctness guarantees, and critically do not integrate with the verifier.

We advocate for *runtime enforcement as a first-class complement* to the eBPF ecosystem. This project will explore this vision to enable diverse runtime checks that unlocks expressiveness, preserves safety, and contains complexity.

EPASS: Verifier-Cooperative Runtime Enforcement

Our key insight is that some safety properties are difficult or impossible to prove statically, but can be enforced efficiently at runtime. Based on this insight, we propose a new **verifier-cooperative runtime enforcement** approach to systematically transform eBPF programs at load time, instrumenting runtime checks guided by verifier analysis. Critically, it ensures all transformations remain within the verifier pipeline to preserve the existing safety model.

To realize this new paradigm, we will develop EPASS, a kernel-integrated transformation framework for eBPF. At a high level, EPASS extends the load-time pipeline so that an eBPF program can be **lifted** into an SSA-based Intermediate Representation (IR), **transformed** through verifier-informed passes, and **re-verified** before JIT compilation. EPASS will introduce several techniques:

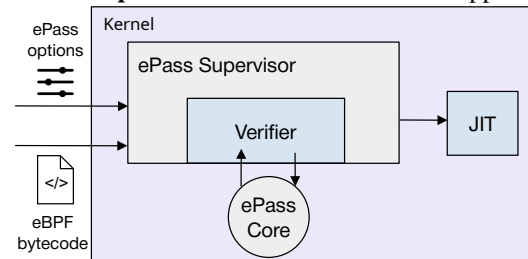


Figure 1: Overview of EPASS.

- **Intermediate Representation (IR):** Instead of instrumenting at bytecode or binary level, which requires significant effort, EPASS raises the level of instrumentation. It introduces a novel SSA-based IR for eBPF that could be transformed from and to eBPF bytecode. This IR features virtual registers, explicit control-flow graphs, verifier annotations, *etc.* This enables systematic analysis and flexible transformation of eBPF programs.
- **Modular Passes:** EPASS provides intuitive APIs for developers to add transformation in modular *passes*. Each pass defines a type of transformation at the IR level. This modular architecture supports a wide range of runtime checks. Developers can author concise, targeted passes to support new use cases and address new safety concerns. After running the passes, EPASS converts the transformed IR back to eBPF bytecode.
- **Verifier Integration:** EPASS transforms programs inside the verifier pipeline, not bypassing it. The instrumentation is done in a way that remains visible and analyzable to the verifier. Furthermore, EPASS maintains *Verifier Information Map* (VI-map) capturing inferred ranges, types, and provenance during verification. It exposes these verifier analysis results to the transformation process, making optimizations and resolution of verifier errors feasible.
- **Safe Instrumentations:** EPASS ensures correct instrumentation through its IR features such as virtual registers and its algorithm to preserve the stack layouts. Moreover, EPASS enforces that all transformed programs are *re-verified* by the verifier, preserving existing safety rules while enforcing additional runtime safety properties.

EPASS comprises two major components shown in Figure 1: (1) **EPASS core**, a small compiler framework that provides APIs to manipulate the eBPF bytecode; (2) **EPASS supervisor**: a component that resides within the kernel, facilitating interactions between EPASS core and the eBPF verifier. EPASS follows the existing eBPF trust model that distrusts users. All transformation logic is implemented within the kernel, and only administrators may add transformation passes. Regular users can choose pre-defined passes but cannot add arbitrary transformation logic. Note that while EPASS introduces additional components, it does not increase the trusted computing base (TCB)—the core TCB is still the verifier. The runtime checks added will pass the verifier, thus enhancing safety.

EPASS includes a rich set of built-in transformation passes that demonstrate the breadth of verifier-cooperative runtime enforcement. These passes fall into two categories: (1) repairing verifier false rejections to improve programmability, and (2) strengthening runtime safety beyond static verification. For example, many programs are rejected because the verifier cannot prove that a dynamic index is within array or map bounds. EPASS’s *bound-repair/masking* pass can automatically insert a guard; the verifier often rejects loops with data-dependent bounds, even when they are safely bounded in practice. EPASS can rewrite such loops by inserting an iteration counter and exit guard; Programs that reuse overlapping stack slots or apply pointer arithmetic to stack addresses often fail verification due to imprecise alias analysis. EPASS can rewrite such ambiguous stack usage into canonical forms. For (2), as some examples, the *counter pass* instruments lightweight instruction counters that enforce execution budgets at runtime; to address verifier blind spots around uninitialized stack reads, the *MSan* pass adds shadow state that detects and rejects unsafe accesses at runtime; for indirect tail calls or computed branches, the *CFI pass* can insert validation stubs.

Preliminary Results. We have developed a prototype of EPASS, open sourced at <https://github.com/OrderLab/ePass>. This prototype demonstrates the feasibility of the proposed approach. It already implements *12 transformation passes*, spanning verifier repair, runtime enforcement, and optimization/portability. Evaluation on 504 real-world programs and 23 user-reported rejected cases showed that EPASS resolved 91% of previously rejected but safe programs, enabling developers to run natural eBPF code without workarounds; mitigated 14 known vulnerabilities, such as uninitialized stack reads and unsafe helper calls. The performance overhead was less than 2% for high-throughput XDP workloads. The IR and framework are lightweight, with most passes requiring fewer than 100 lines of code.

Expected Outcomes

Over the next year, we will evolve EPASS from a prototype into a robust and extensible infrastructure. The expected outcomes will be a production-ready, well-documented EPASS release supporting recent Linux kernels, with a curated library of more than 30 passes. We will also write at least two blog posts and publish papers about EPASS at top systems conferences like OSDI/SOSP. For schedule, in months 1–2, we will extend the IR, stabilize kernel integration, add more tests, and release an updated version. In months 3–4, we will complete repair-oriented passes and evaluate them across a large program corpus to demonstrate substantial programmability gains. In months 5–7, we will finalize runtime safety passes and demonstrate their value through mitigating real-world vulnerabilities. In months 8–9, we will introduce performance optimizations for minimizing runtime overhead and an administrator policy framework enabling configurable pass bundles. In months 10–12, we will implement compatibility shims, polish usability, and engage the community through documentation, blog posts, and kernel RFC to solicit maintainer feedback.